Entity resolution & link enrichment — Fundable case study

Siroj Dushaev · November 2026

1. What I built

Result on the visible set: 95.5 % (organizations 97.1 %, people 92.4 %). 159 of 160 match/dedupe decisions are right, and links are 96–100 % precise. I also ran a cold-cache copy of the input with new ids, shuffled rows and 10 % of API calls failing on purpose. It scored 95.6 % in 172 s for $0.25 plus 559 Serper queries, with all 160 decisions right; earlier cold runs scored 93.8–96.3 %. A rerun from the shipped cache makes no live calls, takes under a second and costs $0.

```
solve.py                uv run solve.py --db DIR --input DIR --out DIR   (writes submission_*.csv + run_report.json)
er/normalize.py         name keys, URL canonicalizers (grader-compatible), DB link index, union-find   [pure, tested]
er/clients.py           one caller for every API: SQLite cache, retry/backoff, deadline, spend ledger
er/llm.py · search.py   OpenRouter strict-JSON chat + embeddings · Serper search + homepage fetch (own sites only)
er/candidates.py        blocking: name/nickname/slug keys, fuzzy, embeddings, hard link keys, website-conflict signal
er/resolve.py           evidence, LLM adjudication with escalation, dedupe inside the input
er/links.py · prompts.py  targeted searches -> LLM picks among observed URLs -> thresholds · prompts and JSON schemas
cache/cache.sqlite      every search/LLM/fetch response behind the shipped submission (no answer-key data)
submissions/            the two CSVs + run_report.json: per row decision, confidence, reason, candidates, link options, spend
scripts/holdout_sim.py  rebuilds the visible set with new ids and shuffled rows for cold-cache tests · tests/ pytest
```

Pipeline. Each type runs with all rows concurrent, and orgs and people run in parallel.

1. **Evidence.** For an organization, a cheap LLM "profile" call proposes the current name, former names, a likely domain and a disambiguating Google query. I run one search, then fetch up to five candidate homepages and keep the ones whose title names the entity. For a person, the LLM extracts role and organization from the article, followed by one search.
2. **Blocking**, tuned for high recall at low cost. Candidates come from:
   - name keys: exact, compact and nickname-aware, plus rapidfuzz fuzzy matching;
   - description embeddings, which catch renames;
   - **profile slugs**: Crunchbase/LinkedIn slugs and X handles keep former names (Ørsted's Crunchbase slug is `dong-energy`), so they work as a free alias table;
   - **hard keys**: a URL in the input's evidence that equals a DB record's domain, LinkedIn, Crunchbase or X.

   A **conflict** signal is added when the input's verified website differs from a same-name candidate's.
3. **Adjudication.** `gpt-5.6-luna` (low reasoning effort) sees the input, its evidence and up to 8 candidates with their signals. It returns `{match_id | NEW, confidence, reason}` under a strict JSON schema, and any id not in the candidate list is rejected. `gpt-5.4` re-checks four cases: confidence below 0.7; a match across unrelated names with no hard key (a likely competitor); a match whose website differs from a fetched site that names the input (a likely namesake), where it also gets the candidate's homepage to compare; and NEW despite a same-name candidate (a possible missed match).
4. **Dedupe.** Candidate pairs come from name keys, fuzzy names, embeddings or a shared website. The LLM confirms each pair, then union-find builds the groups. A twin of a matched row takes that row's match; a NEW group points `dup_of` at its first row.
5. **Links,** found once per NEW group. Three kinds of evidence go in:
   - name queries per site;
   - **domain-anchored** queries (`Name "domain" site:linkedin.com/company`), which cut through namesakes because profile pages show the website;
   - the social links on the entity's own homepage.

   The LLM returns an **option number** from URLs that were actually seen, never free text, so it cannot invent a link. A URL already owned by a DB record is never offered. A link is written only above a confidence threshold: 0.6, or 0.95 for X handles seen only in search. Below 0.95 those were wrong 4 times out of 5, while handles linked from the entity's own site are reliable.

Reliability. All network calls go through one caller:

- It checks the cache first; the cache key is a sha256 of the request with auth headers excluded.
- Each provider has its own concurrency limit.
- Timeouts, 408, 429 and 5xx are retried with exponential backoff and jitter, and `Retry-After` is honored. Other 4xx errors fail fast. Serper query patterns that a plan rejects are retried without quotes.
- A soft deadline at 540 s stops retries so the run finishes with what it has.
- A failed call degrades only its own row: a rules fallback for the decision, or a blank link. A crash still writes valid CSVs.
- Homepage fetches that fail for good are cached as failures too, so a rerun from the cache sees the same evidence and reproduces the same output.

Spend comes from `usage.cost` and is capped by `LLM_BUDGET_USD`. Total development spend was $1.50 of $20 and about 2,400 Serper queries.

Remaining errors (visible set):

- Andrew Stevenson: the model matched a namesake with 0.63 confidence, and the call flips between runs.
- Goldman Sachs: Crunchbase has two slugs for it.
- 10T Holdings: renamed to 50T, and the picker chose a different fund.
- One X handle where the key has none.

2. Production design: a continuous stream of name + text records

Flow. News and scrapers publish to Pub/Sub `mentions` (type, source, source_id, text). The raw payload goes to GCS, keyed by content hash. **Resolver workers** on Cloud Run scale with the backlog. They write to Postgres (`entity`, `entity_alias`, `entity_link`, `mention`, an append-only `decision` log), which feeds BigQuery and the API. Enrichment has its own queue, so matching never waits for search.

Matching and dedupe: a cost ladder, cheapest first, stopping as soon as a level is confident.

Level	What it does	Latency / cost
L0	Unique indexes on normalized domain / LinkedIn / Crunchbase / X, plus `entity_alias` (former names, slugs, reviewer-confirmed aliases)	~ms, $0. Most repeat mentions stop here
L1	Blocking: pg_trgm on names + pgvector ANN on description embeddings, top-k ≤ 8	~20 ms
L2	Cheap LLM with search evidence, only when L0/L1 are inconclusive	~2–4 s, about $1.6 per 1k rows including links
L3	Strong model for the risky cases (cross-name match, website conflict, same-name NEW, low confidence)	~10 % of L2 traffic
L4	Human review	uncertain band only

In a stream, dedupe is matching against **provisional entities**. A NEW decision creates the entity at once, under an advisory lock on its blocking key so two workers can't create the same company at the same moment. Its twin then matches it through L0/L1 with no separate dedupe step. A nightly batch pass re-pairs the day's provisional entities to catch twins that arrived with no shared key.

Enrichment. It runs only for NEW or stale entities. Every link is stored with its source URL, evidence, confidence and `observed_at`, so each datapoint has a source. A uniqueness constraint ties each LinkedIn / Crunchbase / X URL to one entity. A collision is a strong sign of a wrong merge or split, so it goes to review instead of overwriting.

Reliability.

- The idempotency key is source + source_id + content hash, so redelivery and retries are no-ops.
- Each provider gets a rate limit through Cloud Tasks queues or token buckets.
- Calls retry with backoff, and a message goes to a DLQ after N attempts.
- Replay means republishing from GCS under a new `resolver_version` (prompt + model + thresholds).
- A merge is a `merged_into` pointer, never a delete, so any merge can be undone and its mentions re-pointed from the decision log.

Cost and latency. Measured here on the LLM path, enrichment included: $0.26 per 160-row run (about $1.6 per 1k rows) and about 3.5 Serper queries per row, with the cache keyed by content hash. Backfills can use OpenRouter `:batch` models at half price. In production most traffic should stop at L0. The LLM path has a p50 of a few seconds; L0 takes under 100 ms.

Human review. The queue holds:

- rows in the confidence band of roughly 0.5–0.8;
- the risky cases;
- link collisions;
- high-impact entities (large rounds, top investors).

The reviewer sees exactly what the model saw: candidates, signals, evidence URLs and its reason. `run_report.json` is that payload today.

How corrections feed back:

1. **Immediately.** A reviewer decision writes must-link / cannot-link pairs and aliases that L0 applies to the next mention, so a fixed error does not recur and nothing needs retraining.
2. **Evaluation.** Each correction becomes a labelled row in a regression set graded like `grade.py`. A new prompt, model or threshold ships only if it doesn't regress, and the escalation band and per-field link thresholds are recalibrated from reviewed outcomes.
3. **Repair.** A merge or split re-resolves the affected mentions through the decision log and a GCS replay.

Monitoring. NEW-rate and match-rate drift per source, link-collision rate, review-queue age, DLQ depth, cost per 1k records, and freshness from mention to served record.